

TECH CHALLENGE — FASE 2

Otimização de Modelos de Diagnóstico

Algoritmos Genéticos + Interpretação com LLM local

FIAP Postech — IA para Devs

INTEGRANTES DO GRUPO

Otaviano Montes Zibetti — RM374369 — otavianomontes@gmail.com

Felipe Squarizi — RM374950 — fesqua@yahoo.com.br

Fase: 2 — Projeto 1: Otimização de Modelos de Diagnóstico

Data de entrega: Agosto de 2026

Documento de Entrega — Fase 2

Este documento reúne os entregáveis do Tech Challenge da Fase 2. O projeto escolhido foi o **Projeto 1 — Otimização de Modelos de Diagnóstico**: otimização dos hiperparâmetros dos modelos da Fase 1 por **algoritmo genético**, somada à **integração com um LLM local** para interpretar os diagnósticos em linguagem clínica.

Links do projeto

Recurso	URL
Repositório GitHub	https://github.com/otaviano/fiap-tech-challenge-fase2
Vídeo de demonstração	https://youtu.be/z1GaoESr23E
Relatório técnico	https://github.com/otaviano/fiap-tech-challenge-fase2/blob/main/RELATORIO_TECH_CHALLENGE_FASE2.md
Notebook de demonstração	https://github.com/otaviano/fiap-tech-challenge-fase2/blob/main/notebooks/demo.ipynb
Arquitetura	https://github.com/otaviano/fiap-tech-challenge-fase2/blob/main/docs/arquitetura.md
Escalabilidade e monitoramento	https://github.com/otaviano/fiap-tech-challenge-fase2/blob/main/docs/escalabilidade.md
Segurança e produção	https://github.com/otaviano/fiap-tech-challenge-fase2/blob/main/docs/seguranca.md
Infraestrutura como código	https://github.com/otaviano/fiap-tech-challenge-fase2/tree/main/infra
Projeto da Fase 1 (base)	https://github.com/otaviano/fiap-tech-challenge-fase1

Entregáveis do repositório

Item exigido	Onde encontrar
Código-fonte completo	<code>src/diag_opt/</code> — pacote Python modular instalável
Documentação da API	<code>src/diag_opt/serving/api.py</code> + Swagger em <code>/docs</code>
Scripts e notebooks de demonstração	<code>notebooks/demo.ipynb</code> , <code>experiments/</code> , <code>scripts/demo_api.sh</code>
Arquivos de configuração para implantação	<code>Dockerfile</code> , <code>infra/</code> (Terraform)
Relatório técnico	<code>RELATORIO_TECH_CHALLENGE_FASE2.md</code> (reproduzido a seguir)
Vídeo de demonstração (≤ 15 min)	https://youtu.be/z1GaoESr23E

Mapeamento das entregas técnicas exigidas

1. Otimização via Algoritmos Genéticos

Requisito do desafio	Atendido em
Codificação adequada dos hiperparâmetros (representação de genes)	<code>models.py</code> (GeneSpec) e <code>ga/encoding.py</code> — genes <code>float</code> (linear e log), <code>int</code> e categórico; relatório §2.1
Operadores de seleção, cruzamento e mutação	<code>ga/operators.py</code> — torneio, roleta e ranqueamento; crossover uniforme e de um ponto; mutação gaussiana e categórica, com elitismo; relatório §2.2
Função fitness baseada em métricas de desempenho	<code>ga/fitness.py</code> — $0,6 \cdot \text{recall} + 0,3 \cdot \text{F1} + 0,1 \cdot \text{ROC-AUC}$ sobre a classe maligna, em validação cruzada estratificada de 5 folds; relatório §2.3
Comparação entre modelos otimizados e originais	<code>evaluation.py</code> , notebook células 3 e 4; relatório §3
Ao menos 3 experimentos com configurações diferentes	<code>experiments.py</code> — 5 experimentos ; evidência em <code>results/experiments_summary.json</code> ; relatório §2.4

2. Escalabilidade automática, monitoramento e logging

Requisito do desafio	Atendido em
Monitoramento e logging para tracking de desempenho	<code>monitoring/</code> — logs estruturados por geração e tracking em JSON; evidência de execução completa em <code>results/logs/exemplo_execucao.log</code> ; relatório §5
Documentação de arquitetura e decisões	<code>docs/arquitetura.md</code> (com diagrama), <code>docs/escalabilidade.md</code> , <code>docs/seguranca.md</code>
Implementação em nuvem (opcional, pontuação extra)	<code>infra/</code> — Terraform para ECS Fargate com auto-scaling por CPU e por requisições, ALB e CloudWatch

3. Integração com LLMs para interpretação de resultados

Requisito do desafio	Atendido em
Explicações em linguagem natural dos diagnósticos	<code>llm/interpreter.py</code> — Resumo, Fatores, Recomendação e Aviso; relatório §4.1
Transformar dados numéricos em insights acionáveis	<code>llm/interpreter.py</code> — as features são traduzidas em desvios-padrão em relação à média da base, com as <code>top_k</code> mais influentes do caso
Preparar a base para a integração da Fase 3	LLM servido por endpoint OpenAI-compatível, plugável e configurável por ambiente
Prompt engineering	<code>llm/prompts.py</code> — papel de apoio (nunca substituto), restrições explícitas, estrutura de saída fixa e grounding nos dados do caso; relatório §4.2
Avaliação da qualidade das interpretações	<code>llm/quality.py</code> — critérios objetivos de estrutura, segurança clínica, prudência e grounding; relatório §4.3

4. Código e organização

Requisito do desafio	Atendido em
Projeto Python estruturado com ambiente virtual	<code>pyproject.toml</code> , <code>requirements.txt</code> , <code>venv</code> ; pacote instalável com CLI <code>diag-opt</code>
Documentação detalhada, incluindo diagramas de arquitetura	<code>README.md</code> e <code>docs/arquitetura.md</code> (diagrama Mermaid)
Testes automatizados	<code>tests/</code> — 33 testes, 86% de cobertura
IaC para provisionamento (nuvem)	<code>infra/</code> — Terraform

Conteúdo do vídeo (≤ 15 minutos)

Item exigido	Onde aparece
Demonstração do sistema em execução	Execução dos experimentos do GA no terminal e do notebook
Explicação dos diferentes componentes da solução	Estrutura do pacote, <code>docs/arquitetura.md</code> e o bloco dedicado ao GA por dentro
Resultados da otimização via algoritmos genéticos	Curva de convergência, tabela dos 5 experimentos e comparação com os modelos originais
Demonstração da integração com LLMs	<code>diag-opt interpret</code> , interpretação gerada e avaliação de qualidade

Como reproduzir

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt && pip install -e .

diag-opt experiments --model SVM          # 5 experimentos do GA
diag-opt optimize --model GradientBoosting
diag-opt interpret --index 0              # interpretação via LLM local (com fallback)

uvicorn diag_opt.serving.api:app --port 8000 # API REST
pytest --cov=diag_opt                      # 33 testes
```

O algoritmo genético usa semente fixa: a mesma configuração reproduz o mesmo cromossomo e as mesmas métricas entre máquinas e versões de Python.

Relatório Técnico — Tech Challenge Fase 2

FIAP Postech — IA para Devs Projeto 1 — Otimização de Modelos de Diagnóstico via Algoritmos Genéticos + LLM

1. Introdução e contexto

Na Fase 1, desenvolvemos um sistema de suporte ao diagnóstico oncológico (câncer de mama, dataset Wisconsin Breast Cancer) que comparou 6 modelos de Machine Learning, elegendo o **SVM** como melhor (98,25% de accuracy, recall da classe maligna de 97,6%, apenas 1 falso negativo).

Nesta Fase 2, evoluímos a solução em duas frentes:

- 1. **Otimização de hiperparâmetros via Algoritmo Genético (GA)**, comparando os modelos otimizados com os originais da Fase 1;
- 2. **Integração com um LLM local** para traduzir os diagnósticos numéricos em interpretações clínicas acionáveis, com técnicas de *prompt engineering* e avaliação objetiva de qualidade.

Todo o dataset e a convenção clínica da Fase 1 são preservados: a **classe positiva é MALIGNO** (`pos_label=0`), pois o falso negativo (maligno classificado como benigno) é o erro de maior custo clínico.

2. Otimização via Algoritmo Genético

2.1 Codificação dos genes (representação dos indivíduos)

Cada **indivíduo** é um conjunto de hiperparâmetros representado como um dicionário `{hiperparâmetro: valor}` — cada par é um **gene**. Optamos por uma representação **fenotípica** (valores reais/categóricos diretos) em vez de binária, pela legibilidade e mapeamento 1:1 com o modelo.

Cada gene é descrito por um `GeneSpec` (`models.py`) com três tipos:

Tipo	Descrição	Exemplo
<code>float</code>	real, com opção de escala logarítmica	<code>C</code> ∈ <code>[10⁻², 10³]</code> (log)
<code>int</code>	inteiro em faixa	<code>n_estimators</code> ∈ <code>[50, 400]</code>
<code>cat</code>	categórico	<code>kernel</code> ∈ <code>{rbf, poly, sigmoid}</code>

A escala logarítmica é essencial para hiperparâmetros como `C`, `gamma` e `learning_rate`, cujas ordens de grandeza importam mais que o valor absoluto.

Espaços de busca por modelo:

- **SVM:** `C` (log), `gamma` (log), `kernel` (categórico)

- **RandomForest:** `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features` (categórico misto), `bootstrap` (booleano, modelado como categórico `{True, False}`)
- **GradientBoosting:** `n_estimators`, `learning_rate` (log), `max_depth`, `subsample`

2.2 Operadores genéticos

Implementados em `ga/operators.py`:

- **Seleção:** três métodos — por **torneio** (pressão seletiva ajustável por `k`), por **roleta** (proporcional ao valor do fitness) e por **ranqueamento** (proporcional à posição no ranking, pesos 1...N);
- **Crossover:** **uniforme** (cada gene herdado de um pai com prob. 0,5) e de **um ponto**;
- **Mutação:** gene a gene, com **perturbação gaussiana** para genes numéricos (busca local) e reamostragem para categóricos, sempre respeitando o domínio;
- **Elitismo:** os melhores indivíduos passam intactos entre gerações, garantindo que o melhor fitness nunca regrida.

***Por que implementar roleta e ranqueamento?** Nosso fitness é uma combinação de métricas de classificação, então praticamente toda a população vive num intervalo estreito (ex.: 0,90–0,97). Na roleta, um indivíduo com fitness 0,97 recebe apenas ~8% mais chance que um com 0,90 — a pressão seletiva quase desaparece. O ranqueamento é invariante à escala: o melhor sempre recebe peso `N` e o pior peso `1`, independentemente de a diferença absoluta entre eles ser 0,001 ou 0,5. Os dois estão nos experimentos justamente para tornar esse efeito mensurável.*

2.3 Função fitness

A fitness (`ga/fitness.py`) é uma **combinação ponderada de métricas**, estimada por **validação cruzada estratificada (5-fold)** — a mesma estratégia da Fase 1, que evita sobreajuste a uma única partição:

```
fitness = 0,6 × recall(maligno) + 0,3 × F1(maligno) + 0,1 × AUC
```

O peso dominante no recall reflete a prioridade clínica. Os pesos são configuráveis (parametrizados nos experimentos). Um **cache** evita reavaliar cromossomos repetidos, reduzindo drasticamente o custo. O **test set fica reservado** apenas para a comparação final (sem vazamento de dados).

2.4 Experimentos realizados

Foram executadas **5 configurações** distintas do GA (requisito: ≥ 3), variando população, taxa de mutação, método de seleção e tipo de crossover. Cada uma altera **poucas variáveis por vez** em relação à `baseline_ga`, para que o efeito de cada fator seja atribuível:

Experimento	População	Gerações	Taxa mutação	Seleção	Crossover	O que isola
baseline_ga	20	15	0,15	torneio	uniforme	referência
high_mutation	20	15	0,35	torneio	uniforme	exploração vs. aproveitamento
large_population	40	10	0,15	torneio	uniforme	diversidade por geração
roulette_onepoint	20	15	0,15	roleta	1 ponto	outro regime de busca
rank_selection	20	15	0,15	ranqueamento	uniforme	só o método de seleção

Resultados (SVM, modelo vencedor da Fase 1) — fitness por validação cruzada:

Experimento	Best fitness (CV)	Melhores hiperparâmetros	Avaliações	Tempo
baseline_ga	0,9726	C=2,61; γ =0,034; rbf	99	4,7 s
high_mutation	0,9655	C=12,46; γ =0,005; rbf	154	5,0 s
large_population	0,9655	C=30,95; γ =0,003; rbf	156	5,4 s
roulette_onepoint	0,9762	C=5,31; γ =0,034; rbf	144	4,8 s
rank_selection	0,9655	C=30,95; γ =0,003; rbf	113	4,3 s

Observações sobre os experimentos:

- A configuração `roulette_onepoint` atingiu o maior fitness de CV; `baseline_ga` ficou muito próximo com **menos avaliações** (99) — ou seja, mais eficiente.
- `high_mutation` explorou mais o espaço (154 avaliações, valores de `C` mais altos), mas não superou as configurações mais conservadoras — sinal de que, para o SVM neste dataset, exploração excessiva não compensa. É a confirmação empírica do compromisso clássico **exploração × aproveitamento**: aumentar a mutação amplia a área varrida, mas dilui a convergência.
- `rank_selection` é o experimento mais controlado: muda **apenas** o método de seleção em relação à `baseline_ga` (mesma população, gerações, mutação e crossover). O resultado permite **medir a pressão seletiva de cada método**. Como o cache só conta cromossomos *distintos*, o número de avaliações é um indicador direto de diversidade: quanto mais pressão seletiva, mais a população se repete e menos avaliações novas acontecem. Com população 20 e 15 gerações, mantendo tudo o mais constante:

Seleção	Avaliações	Diversidade	Fitness final
torneio (k=3)	99	menor	0,9726
ranqueamento	113	intermediária	0,9655
roleta ¹	144	maior	0,9762

¹ a roleta roda com crossover de 1 ponto e `crossover_rate` 0,9, então não é uma comparação perfeitamente controlada — mas a ordem de grandeza é clara.

A ordenação bate exatamente com a teoria: no **torneio com k=3**, o melhor indivíduo tem 15% de chance por sorteio (3/20) — é o mais elitista dos três. No **ranqueamento linear**, o melhor recebe peso 20 de um total de 210 ≈ 9,5% — pressão moderada e, crucialmente, **estável**. Na **roleta**, com o fitness comprimido entre 0,90 e 0,97, as probabilidades ficam quase uniformes — quase não há pressão. Ou seja: torneio > ranqueamento > roleta em pressão seletiva, e o inverso em diversidade mantida.

A lição prática é que **nenhum dos três domina**: o torneio converge mais rápido e chegou a um fitness alto; a roleta, justamente por explorar mais, tropeçou no melhor fitness de todos; e o ranqueamento fica no meio-termo. Em um espaço pequeno como este (3 hiperparâmetros do SVM), essa diferença é ruído; em espaços maiores, é o ranqueamento que evita tanto a convergência prematura do torneio quanto a estagnação da roleta. - O cache manteve o número de avaliações bem abaixo do total teórico (pop × gerações = 300 para `baseline_ga`, contra 99 efetivas). - `baseline_ga` é a configuração *default* do `GACONFIG` — é ela que roda no `notebooks/demo.ipynb` e no comando `diag-opt optimize` sem argumentos. As demais são reproduzidas por `experiments/run_experiments.py`.

Atenção à leitura: *fitness de CV mais alto não implica melhor desempenho no test set — ver §3.1, onde `baseline_ga` tem fitness maior que `high_mutation` e ainda assim fica atrás no conjunto de teste.*

3. Comparativo: modelos originais vs. otimizados

A comparação final é feita **no test set** (20% dos dados, nunca visto pelo GA).

3.1 SVM — o modelo já era quase ótimo

O SVM da Fase 1 (`C=1,0`, `gamma='scale'`, `rbf`) atingiu no test set: accuracy 0,9825, recall(maligno) 0,9762, F1(maligno) 0,9762, **1 FN e 1 FP**. Abaixo, o que cada uma das 5 configurações do GA produziu **no mesmo test set**:

Configuração do GA	Fitness (CV)	Accuracy	Recall (mal.)	F1 (mal.)	FN	FP
<i>Original (Fase 1)</i>	—	0,9825	0,9762	0,9762	1	1
<code>baseline_ga</code> (C=2,61)	0,9726	0,9737	0,9762	0,9647	1	2
<code>high_mutation</code> (C=12,46)	0,9655	0,9825	0,9762	0,9762	1	1
<code>large_population</code> (C=30,95)	0,9655	0,9825	0,9762	0,9762	1	1
<code>roulette_onepoint</code> (C=5,31)	0,9762	0,9825	0,9762	0,9762	1	1
<code>rank_selection</code> (C=30,95)	0,9655	0,9825	0,9762	0,9762	1	1

Leitura honesta do resultado: nenhuma configuração **superou** o SVM da Fase 1 no test set. Quatro delas o **igualam** em todas as métricas; a `baseline_ga` fica ligeiramente **abaixo** (um falso positivo a mais, -0,9 pp de accuracy, -1,2 pp de F1), apesar de ter fitness de CV maior que `high_mutation` e

`large_population` — um caso clássico de pequena diferença de CV que não se traduz em ganho no conjunto de teste (114 amostras: 1 erro = 0,88 pp).

Vale destacar o gene categórico: o GA tinha `kernel ∈ {rbf, poly, sigmoid}` disponível e escolheu **rbf nas cinco configurações**, sem exceção. Ou seja, a busca não validou apenas os valores de `C` e `gamma` — validou também a decisão estrutural de qual família de fronteira de decisão usar, que a Fase 1 havia tomado pelo default do scikit-learn.

O recall do maligno — a métrica que priorizamos — e o número de falsos negativos (1) permanecem inalterados em **todas** as configurações. A conclusão é que o SVM com hiperparâmetros *default* já estava praticamente no ótimo para este dataset: o GA **confirma** a escolha da Fase 1 em vez de melhorá-la. Tratamos isso como resultado positivo de validação — o método não "inventa" ganhos onde não há espaço para eles.

Configuração servida em produção: a API usa `C=5,31`, `gamma=0,0336`, `rbf` (`roulette_onepoint`, maior *fitness* de CV) — ver `serving/api.py`. Já o `GAConfig()` default, usado pelo `notebooks/demo.ipynb` e pelo comando `diag-opt optimize`, corresponde à `baseline_ga` — por isso o notebook reproduz a linha `C=2,61` desta tabela, e não a de produção.

3.2 Gradient Boosting — troca de erro favorável ao contexto clínico

Para exercitar o GA onde havia espaço, aplicamo-lo a um modelo com **baseline mais fraco** na Fase 1 (Gradient Boosting, recall CV de 0,906):

Métrica	Baseline (params default)	Otimizado (GA)	Δ
Fitness (CV)	0,9216	0,9655	+0,044
Accuracy (test)	0,9561	0,9561	0,000
Recall (maligno, test)	0,9048	0,9286	+0,024
Falsos Negativos (test)	4	3	-1
Falsos Positivos (test)	1	2	+1

O que realmente aconteceu: a accuracy no test set é **idêntica** — são 5 erros em 114 amostras nos dois casos. O GA não reduziu o total de erros; ele **redistribuiu** os erros, trocando um falso negativo por um falso positivo.

Isso é uma melhora **para o nosso critério**, não uma melhora absoluta. A função de fitness pondera `recall(maligno)` com peso 0,6 justamente porque, em oncologia, os dois erros não têm o mesmo custo: um falso negativo libera um paciente com tumor maligno sem tratamento; um falso positivo aciona exames adicionais em um caso benigno. O GA otimizou exatamente aquilo que lhe pedimos — e o efeito visível é essa troca. Hiperparâmetros encontrados: `n_estimators=294`, `learning_rate=0,155`, `max_depth=3`, `subsample=0,54`.

Vale registrar a escala: mesmo otimizado, o Gradient Boosting (recall 0,9286, 3 FN) permanece **inferior ao SVM da Fase 1** (recall 0,9762, 1 FN). O ganho demonstrado aqui é do *método*, não do sistema de diagnóstico entregue.

3.3 Random Forest

Métrica	Baseline (params default)	Otimizado
Fitness (CV)	0,9444	0,9497
Accuracy (test)	0,9561	0,9561
Recall (maligno, test)	0,9286	0,9286
Falsos Negativos (test)	3	3
Falsos Positivos (test)	2	2

Ganho marginal no fitness de CV (+0,005) que **não se traduz em nenhuma mudança no test set** — as métricas são idênticas. Hiperparâmetros encontrados: `n_estimators=96`, `max_depth=26`, `min_samples_split=3`, `min_samples_leaf=2`, `max_features='log2'`, `bootstrap=True`.

Um detalhe informativo do gene booleano: o GA testou `bootstrap=False` (floresta sem *bagging*, cada árvore treinada sobre a amostra completa) e **manteve `True`** no melhor indivíduo. Ou seja, a amostragem com reposição — o mecanismo que dá o "Random" ao Random Forest — se confirma útil neste dataset, em vez de ser assumida por convenção.

Nota metodológica sobre os baselines de GB e RF: a Fase 1 avaliou no test set apenas o modelo vencedor (SVM); para os demais modelos, publicou somente validação cruzada. As colunas "baseline" de §3.2 e §3.3 foram, portanto, **recalculadas nesta Fase 2** treinando GB e RF com os hiperparâmetros default do scikit-learn, no mesmo split (`random_state=42`) da Fase 1. Os valores de CV batem com os publicados na Fase 1 (GB recall 0,906; RF 0,939), o que confirma que o split e o pipeline foram preservados. O "Fitness (CV)" também é uma métrica desta fase — a combinação $0,6 \cdot \text{recall} + 0,3 \cdot F1 + 0,1 \cdot \text{AUC}$ não existia na Fase 1.

3.4 Síntese

O balanço final da otimização é o seguinte:

Modelo	Ganho no test set	Interpretação
SVM	Nenhum (empate, ou leve piora na <code>baseline_ga</code>)	Confirma a escolha da Fase 1
Gradient Boosting	Troca de 1 FN por 1 FP (accuracy igual)	Ganho no critério clínico priorizado
Random Forest	Nenhum	Modelo já estável no seu patamar

Nenhum modelo otimizado superou o SVM da Fase 1, que segue sendo o modelo servido em produção. O resultado desta fase não é um salto de performance preditiva — é a **validação científica** da solução anterior (o GA, buscando livremente em um espaço amplo, não encontrou nada melhor) somada à demonstração de que o método captura o critério clínico correto quando há espaço para agir (o caso do Gradient Boosting). Em um dataset de 569 amostras com o SVM já em 97,6% de recall, esse é o resultado que a honestidade metodológica permite reportar.

4. Integração com LLM para interpretação de resultados

4.1 Abordagem e decisão de arquitetura

Utilizamos um **LLM local open-source: Qwen3 4B Instruct**, no arquivo `Qwen3-4B-Instruct-2507-Q4_K_M.gguf` (4 bilhões de parâmetros, quantização Q4_K_M), servido por `llama.cpp` com API compatível com a da OpenAI.

A decisão é central e clínica: **dados de pacientes não saem da rede local**, alinhado à **LGPD**, com custo de API zero. A quantização em 4 bits é o que torna isso viável — o modelo roda em CPU ou em uma GPU modesta, sem exigir infraestrutura dedicada, que é o cenário realista de um hospital.

O endpoint é plugável por variável de ambiente (`LLM_BASE_URL`, `LLM_MODEL`), aceitando Ollama, LM Studio, vLLM etc. — trocar de modelo não exige mudança de código.

Fluxo de interpretação (`llm/interpreter.py`):

1. Constrói o **contexto do paciente**: predição, confiança (probabilidade calibrada) e as **características celulares mais relevantes**, medidas em desvios-padrão em relação à distribuição de treino;
2. Monta o prompt e chama o LLM;
3. Se o LLM estiver indisponível, usa um **fallback determinístico** (template), garantindo degradação graciosa.

4.2 Prompt engineering

Técnicas aplicadas (`llm/prompts.py`):

- **Role prompting**: o modelo assume o papel de assistente de apoio (nunca substituto) ao médico;
- **Restrições explícitas**: proibição de diagnóstico definitivo, uso de linguagem probabilística, reforço de que a decisão é do profissional;
- **Estrutura de saída fixa**: seções *Resumo* / *Fatores* / *Recomendação* / *Aviso*, fáceis de auditar;
- **Grounding**: injeção apenas dos números do caso, reduzindo alucinação.

Exemplo de saída real do modelo (caso maligno, confiança 95,3%):

```
## Resumo — O modelo prediz com alta confiança (95,3%) malignidade, com base em padrões celulares consistentes com tumores malignos. ## Fatores que mais influenciaram — perimeter error (2,2  $\sigma$  acima da média, irregularidade da forma), worst perimeter (heterogeneidade celular)... ## Recomendação — considerar biópsia para confirmação... ## Aviso — apoio à decisão, não diagnóstico; a palavra final é do médico.
```

4.3 Avaliação da qualidade das interpretações

Em vez de avaliação apenas subjetiva, definimos **critérios objetivos e verificáveis** (`llm/quality.py`), gerando um score de conformidade reprodutível — útil para comparar prompts:

Critério	O que verifica
Estrutura	presença das seções esperadas
Segurança clínica	disclaimer de "apoio, não diagnóstico"
Prudência	linguagem probabilística
Sem afirmação categórica	ausência de termos como "diagnóstico definitivo", respeitando negações
Grounding	cita as características que embasaram a predição
Idioma/tamanho	PT-BR e extensão adequada

Avaliando **8 casos reais** do test set (4 malignos e 4 benignos), o LLM local atingiu **score 1,00 em todos** — nenhuma reprovação em nenhum critério. O fallback determinístico também atinge 1,00, por construção.

Um falso positivo que o próprio avaliador nos ensinou. Numa versão anterior, o critério *sem afirmação categórica* reprovava com frequência, e a leitura inicial foi de que o modelo era assertivo demais. Investigando o texto reprovado, o trecho responsável era:

"Essa saída é um apoio à decisão clínica e **não constitui diagnóstico definitivo**."

Ou seja: o disclaimer **correto** — exatamente a prudência exigida — continha a substring proibida. A verificação por *substring* não enxergava a negação e penalizava o modelo por acertar. A correção (`_has_categorical_claim`) passou a considerar apenas ocorrências **sem negador na mesma sentença**, distinguindo "não constitui diagnóstico definitivo" (prudente) de "trata-se de um diagnóstico definitivo" (categórico). Dois testes cobrem os dois lados.

Limitação honesta desta métrica. Com o prompt atual, todos os casos atingem a nota máxima — o que significa que o score **não discrimina qualidade informativa**. Ele é um *guarda-corpo*: mede conformidade com garantias mínimas de segurança clínica (estrutura, disclaimer, linguagem probabilística, grounding nos dados do caso). Seu valor está em **detectar regressão** — se alguém alterar o prompt ou trocar o modelo e uma dessas garantias cair, o score acusa imediatamente. Avaliar elegância ou utilidade clínica do texto exigiria julgamento humano ou um LLM-juiz, fora do escopo desta fase.

5. Escalabilidade, monitoramento e nuvem

- **Monitoramento/logging** (`monitoring/`): logging estruturado (console + arquivo rotativo) e *tracking* de cada execução do GA em JSON (configuração, convergência por geração, métricas finais). Cada geração produz uma linha com timestamp, melhor fitness, fitness médio e recall; ao final de cada experimento, uma linha de resumo com fitness, número de avaliações e tempo:

```
2026-08-27 20:10:57,158 | INFO | diag_opt | [SVM_rank_selection] geração 09 | best=0.9655
| mean=0.8730 | recall=0.9576
2026-08-27 20:10:58,331 | INFO | diag_opt | [SVM_rank_selection] concluído |
best_fitness=0.9655 | evals=113 | 4.3s
```

O log completo da execução que gerou os resultados deste relatório está versionado em `results/logs/exemplo_execucao.log` (5 experimentos do SVM + Gradient Boosting + Random Forest). - **Escalabilidade** (`docs/escalabilidade.md`): dois workloads distintos — otimização (batch, paralelizável por indivíduo) e inferência (online, *stateless*, auto-scaling horizontal). - **Rastreabilidade do que está no ar**: o endpoint `GET /health` devolve, além do status usado pelo load balancer, a configuração do modelo servido (SVM, `C=5,31`, `gamma=0,0336`, `rbf`, com o experimento de origem) e o LLM configurado — permitindo auditar qual cromossomo do GA está em produção sem inspecionar o código ou o container. - **Nuvem/laC** (`infra/`): Terraform provisionando ECS Fargate com **auto-scaling** por CPU e por requisições, ALB e CloudWatch — a materialização da arquitetura de escalabilidade (pontuação extra do enunciado).

Arquitetura completa em `docs/arquitetura.md`; segurança e produção em `docs/seguranca.md`.

6. Desafios enfrentados e soluções

Desafio	Solução
SVM da Fase 1 já era quase ótimo — pouco espaço para ganho	Aplicar o GA também a modelos mais fracos (GB) para demonstrar valor; tratar a confirmação do SVM como resultado positivo de validação
Custo computacional do GA (muitas avaliações × CV)	Cache de fitness + <code>n_jobs=-1</code> na CV; redução de ~300 para ~100 avaliações efetivas
<code>SVC(probability=True)</code> deprecado e lento (sklearn 1.9)	Usar <code>decision_function</code> para o AUC no GA; calibrar probabilidades só no modelo de serviço (<code>CalibratedClassifierCV</code>)
Dependência de um LLM externo quebraria testes/demo	Cliente com detecção de disponibilidade + fallback determinístico ; testes forçam o caminho offline
Privacidade de dados de pacientes	LLM local , dados anonimizados no prompt (LGPD)
Qualidade variável das respostas do LLM	Avaliação objetiva automatizada + prompt com restrições explícitas
Modelo <code>qwen3</code> emite blocos de raciocínio <code><think></code>	Limpeza automática da resposta no cliente

7. Como reproduzir

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Experimentos do GA
PYTHONPATH=src python experiments/run_experiments.py --model SVM

# Interpretação via LLM (usa LLM local; cai no fallback se indisponível)
pip install -e . && diag-opt interpret --index 0

# Testes (33 testes, 86% de cobertura)
pytest --cov=diag_opt
```

8. Conclusão

Entregamos um sistema que (1) otimiza hiperparâmetros de modelos de diagnóstico via Algoritmo Genético com codificação, operadores e fitness próprios; (2) valida cientificamente a escolha da Fase 1 (o SVM permanece imbatível no test set, mesmo com busca livre no espaço de hiperparâmetros) e reduz um falso negativo no Gradient Boosting ao custo de um falso positivo — a troca correta para o critério clínico adotado; (3) interpreta diagnósticos em linguagem natural com um LLM local, respeitando privacidade e com avaliação objetiva de qualidade; e (4) documenta arquitetura, escalabilidade, segurança e infraestrutura como código — endereçando diretamente os pontos de melhoria apontados na Fase 1.